

OriPics GPSA v2.2.4

OriPics — Generator Product Security Architecture Document

Applicant: SantaHades Co., Ltd. **Record ID:** 019e4988-9d7c-72f8-8675-22eacf3e1904 **Version:** 2.2.4 (2026-06-19) **Supersedes:** v2.2.3 (2026-05-31), v2.2.2 (2026-05-30), v2.2.1 (2026-05-29), v2.2 (2026-05-29), v2.1 (2026-05-28), v2.0 (2026-05-24), v1.0 (2026-05-22) **Target Assurance Level:** Level 1

Summary of Changes in v2.2.4

v2.2.4 responds to the approver's Sample Evaluation Summary (2026-06-19):

- 1. Declared ingestion format corrected (§C.1.7).** The submission previously declared still-image support as `image/png` only, while `sample-oripics-with-ingredient.png` ingests a JPEG ingredient (a Google Pixel `image/jpeg`). §C.1.7 now declares **claim generation** (the asset OriPics writes) as `image/png`, and **claim ingestion / validation** (a prior-manifest asset the user uploads, preserved as a `parentOf` ingredient) as `image/png` and `image/jpeg`. This reflects actual product behaviour: OriPics ingests PNG or JPEG and always re-encodes its signed output to PNG.
- 2. Internal verifier scope clarified (§C.1.7, consistent with §C.2.4).** The user-facing convenience verifier is pinned to `@contentauth/c2pa-node v0.5.5 (c2pa-rs v0.82.1)`, which does not honor a trusted RFC 3161 timestamp for an *expired ingested ingredient* certificate (a known SDK version lag; current `c2patool / c2pa-rs` resolves such an ingredient to `trusted`). OriPics treats `c2patool / contentcredentials.org` as the authoritative validation reference and will align the internal verifier once a conformant newer SDK build is published upstream (a `c2pa-node v0.6.0` upgrade was attempted but its prebuilt binaries are not yet available). This limitation affects only the convenience verify feature's reporting of ingested ingredients, not claim generation.

No code change to the signing/generation path; sample manifests are unchanged from v2.2.3 (regenerated 2026-06-16 with entity-namespaced custom action parameters).

Summary of Changes in v2.2.3

v2.2.3 remediates the sample nonconformity identified in the C2PA conformance review of v2.2.2 (Scott S. Perry, 2026-05-31). The review confirmed the **architecture passes conformance**; the remaining issue was in the generated sample manifests.

c2pa.actions.v2 (and the other C2PA-standard assertions) are now placed in `created_assertions`, not `gathered_assertions`. This was a previously communicated nonconformity. Root cause: the `c2pa-rs` default places only the hard binding (`c2pa.hash.data`) in `created_assertions` and everything else — including `c2pa.actions`, `c2pa.ingredient`, and thumbnails — in `gathered_assertions`. Neither `builder.addAction()` nor `builder.setIntent('edit')` changes that default placement. The fix is to **configure** the `c2pa-rs` builder setting `created_assertion_labels` (rather than accept its defaults):

```
builder.created_assertion_labels = [
  "c2pa.actions", "c2pa.ingredient",
  "c2pa.thumbnail.claim", "c2pa.thumbnail.ingredient"
]
```

After this change, `c2patool --detailed` reports `c2pa.actions.v2`, `c2pa.ingredient.v3`, and both thumbnails in `created_assertions`; only the vendor-specific `com.oripics.proof` remains in `gathered_assertions` (intentional — it is not a C2PA-standard assertion). See §C.2.4 §2.

Verified with `c2patool 0.26.56` + the C2PA Conformance Trust List on both samples: `validation_state: Valid`; the only failure on the active manifest is `signingCredential.untrusted` (expected sandbox certificate). The ingested Google Pixel ingredient validates as `signingCredential.trusted` + `timeStamp.trusted` (its short-lived per-image certificate is accepted via its trusted timestamp).

Summary of Changes in v2.2.2

v2.2.2 remediates the two nonconformities identified in the C2PA conformance review of v2.2.1 (Scott S. Perry, 2026-05-30):

- 1. Validation logic now consults the C2PA Trust List.** The OriPics verification function (used by the public verify feature) previously did not load the C2PA Trust List when running in production mode, so even a manifest signed by a certificate chaining to the Trust List would be reported as `untrusted`. The verifier now loads the official **C2PA Conformance Trust List** — `C2PA-TRUST-LIST.pem` (signing CAs) and `C2PA-TSA-TRUST-LIST.pem` (timestamp CAs) from `c2pa-org/conformance-public`, plus the ECU trust configuration — via `createTrustSettings()`, with `verifyTrust` and `verifyTimestampTrust` enabled. A signing certificate chaining to the Trust List (e.g. `SSL.com C2PA ECC/RSA Root CA 2025`) now yields a `trusted` verdict.
- 2. Standard-tier (ingested file) action corrected to `c2pa.opened`.** OriPics does not create the asset for a Web upload / file pick — the user supplies it — so the Generator Product has no right to assert `c2pa.created` (which claims creation of the asset itself, not merely of the manifest). The only conformant inception action for an ingested file is `c2pa.opened` (no `digitalSourceType`), with the uploaded file recorded as a **parent0f ingredient of unknown provenance** (OriPics did not create it and cannot prove C2PA creation provenance). This supersedes the v2.2.1 `c2pa.created` decision.

Scenario	Action (v2.2.2, as implemented and as in the attached samples)
Standard tier (Web upload / mobile file pick), no prior manifest	<code>c2pa.opened</code> (no <code>digitalSourceType</code>) + uploaded file as <code>parent0f</code> ingredient of unknown provenance
Verified tier (mobile native camera, App Attest / Play Integrity)	<code>c2pa.created</code> + <code>digitalSourceType: digitalCapture</code>
Any tier, prior C2PA manifest present	<code>c2pa.opened</code> (no <code>digitalSourceType</code>) + prior manifest as <code>parent0f</code> ingredient

Rationale: `c2pa.created` asserts creation of the asset; OriPics may legitimately assert it only for Verified-tier captures, where in-app camera hardware (confirmed by App Attest / Play Integrity) actually produces the asset within the TOE. For all ingested files, OriPics asserts only that it opened an existing file (`c2pa.opened`) and seals it; if creation attribution were ever required, the CAWG identity assertion (in `gathered_assertions`) would be the conformant mechanism — OriPics does not claim creation and therefore does not use it.

Summary of Changes in v2.2.1

This is an editorial consistency correction of v2.2. **The implemented behavior is unchanged** — the v2.2 *Summary of Changes* already specified `c2pa.created` with no `digitalSourceType` for Standard-tier uploads, and the implementation and sample files have always produced exactly this. However, several v2.2 body sections inconsistently described the Standard-tier action as `c2pa.published`, `c2pa.opened`, or `c2pa.created + digitalCapture`. v2.2.1 corrects all body sections, tables, the architecture diagram, and the attachment description to consistently state the actual, conformant behavior:

Scenario	Action (v2.2.1, as implemented and as in the attached samples)
Standard tier (Web upload / mobile file pick), no prior manifest	<code>c2pa.created</code> with no <code>digitalSourceType</code> (no capture claim)
Verified tier (mobile native camera, App Attest / Play Integrity)	<code>c2pa.created</code> + <code>digitalSourceType: digitalCapture</code>
Any tier, prior C2PA manifest present	<code>c2pa.opened</code> + <code>c2pa.ingredient.v3</code> (<code>relationship: "parent0f"</code>)

Rationale: per the C2PA specification, the first action of a parentless manifest must be `c2pa.created` or `c2pa.opened` (a standalone `c2pa.published` first action fails validation with `assertion.action.malformed`). For unverified uploads, OriPics therefore uses `c2pa.created` **without** any `digitalSourceType` — asserting only that OriPics created the C2PA manifest, and making **no claim that the underlying image was captured or otherwise originated by OriPics**. The `digitalCapture` source type is reserved exclusively for Verified-tier mobile captures confirmed by device attestation.

Summary of Changes in v2.2

Item	v2.1	v2.2
<code>c2pa.actions.v2</code> assertion placement	<code>gathered_assertions</code> (via <code>addAssertion()</code>) — nonconformant	<code>created_assertions</code> (via <code>builder.addAction()</code> per action) — conformant

Item	v2.1	v2.2
Web client in TOE boundary	Included as Edge subsystem	Explicitly excluded from TOE — Web is content-submission portal only; Req 6.2.1.5/6 satisfied by mobile clients (App Attest/Play Integrity)
Action for Standard tier (web/mobile file pick)	<code>c2pa.created</code> + <code>digitalCapture</code> — incorrect (capture not verifiable)	<code>c2pa.created</code> (no <code>digitalSourceType</code>); only Verified tier (mobile camera, App Attest/Play Integrity) uses <code>c2pa.created</code> + <code>digitalCapture</code>
Signing key storage language	"SSL.com eSigner Cloud HSM (production)" — tentative/unimplemented	Clearly stated: current production = Vercel encrypted env vars (AES-256); HSM is planned Phase 2 post-conformance
TOE for Req 6.2.1.5/6	Web + Mobile	Mobile only (iOS App Attest + Android Play Integrity); Web explicitly excluded

Summary of Changes in v2.1

Item	v2.0	v2.1
TLS for Backend→Supabase	TLS 1.2+	TLS 1.3 enforced via <code>instrumentation.ts</code> <code>tls.DEFAULT_MIN_VERSION</code>
Pre-existing manifest handling (§C.2.7)	"Pending guidance" (not implemented)	Approach A implemented: <code>c2pa.opened</code> + <code>parentOf</code> ingredient
Action for ingested file	<code>c2pa.created</code> (incorrect)	<code>c2pa.opened</code> when prior manifest detected
Assertion classification note	"All in created_assertions" (incorrect)	Corrected: c2pa-rs Claim v2 places hash in created, data in gathered
Attachments	Standard sample only	+ingredient sample (Pixel Camera JPG from Interoperability Library)

Summary of Changes in v2.0

Item	v1.0	v2.0
Implementation Class	Backend	Distributed
TOE Architecture	Backend-only diagram	Distributed diagram (3 Edge clients + Backend)
Edge-to-Backend mutual auth	Not documented	Documented (§C.2.2 §4)
Digital Source Type	<code>algorithmicMedia</code> (incorrect)	<code>digitalCapture</code>
Capture-only / Ingredient policy	Not documented	New §C.2.7

C.1. Generator Product Information

C.1.1. Applicant Organization Details

- **Legal Name:** SantaHades Co., Ltd. (주식회사 산타하데스)
- **Address:** #B01-H306, Terrace Garden, 150-29 Gongse-ro, Giheung-gu, Yongin-si, Gyeonggi-do 17084, Republic of Korea
- **Contact Person:** Yongseog Son, Representative Director and CEO
- **Email:** hi@ori.pics
- **Phone:** +82-10-3717-9717
- **Security Contact:** security@ori.pics (RFC 9116 security.txt published at <https://www.ori.pics/well-known/security.txt>)

C.1.2. Distinguished Name

Field	Value
Common Name (CN)	OriPics
Organization (O)	SantaHades Co., Ltd.

Field	Value
Organizational Unit (OU)	(not applicable — single-team organization)
Country (C)	KR

C.1.3. Generator Product Description

OriPics is a photo provenance and authenticity platform for the Korean consumer and SMB market, with planned expansion to global users. It allows users to prove that an image is original (untampered) and to certify when and where it was captured.

Platform coverage and TOE classification:

Platform	TOE membership	Role
OriPics iOS	Within TOE	Native camera capture (Verified tier, App Attest) + file pick (Standard tier). Device integrity: Apple App Attest.
OriPics Android	Within TOE	Native camera capture (Verified tier, Play Integrity) + file pick (Standard tier). Device integrity: Google Play Integrity.
OriPics Web	Outside TOE	Browser-based content submission portal only. Browser JS cannot provide cryptographic subsystem authentication. Web uploads are ingested via a <code>c2pa.opened</code> action (no <code>digitalSourceType</code>) with the uploaded file recorded as a <code>parentOf</code> ingredient of unknown provenance — OriPics did not create the asset. Not a conformant Edge subsystem.

TOE boundary justification: The C2PA Generator Product TOE requires that Edge subsystems be mutually authenticated with the Backend (Req 6.2.1.5/6). The mobile clients satisfy this requirement via Apple App Attest (iOS) and Google Play Integrity (Android). The Web client runs in a browser JavaScript environment that provides no secure storage or execution boundary; it cannot be cryptographically authenticated as a subsystem instance. Accordingly, the Web client is **explicitly excluded from the TOE** for C2PA manifest generation purposes. Web uploads are accepted as user-submitted content; the Backend ingests them with a `c2pa.opened` action and records the uploaded file as a `parentOf` ingredient of **unknown provenance** — asserting only that OriPics opened and sealed an existing file, and making no claim that OriPics created, captured, or originated the underlying image.

The mobile and backend clients share a common backend signing service hosted on Vercel Functions. The backend holds the C2PA signing certificate; no signing key material is present on any edge device.

Note on separate submissions: Per C2PA Conformance Program requirements, OriPics iOS and OriPics Android will be submitted as separate generator products with separate Intake Forms. This document covers the full Distributed architecture shared by all platforms. The backend signing service and certificate are common to all platforms; platform-specific details are called out where relevant.

Intended use cases:

- Traffic accident scene documentation for insurance and legal submissions
- Real estate move-in/move-out condition evidence
- Creator portfolio originality proof
- Media and journalism photo provenance
- Dating and social media identity verification

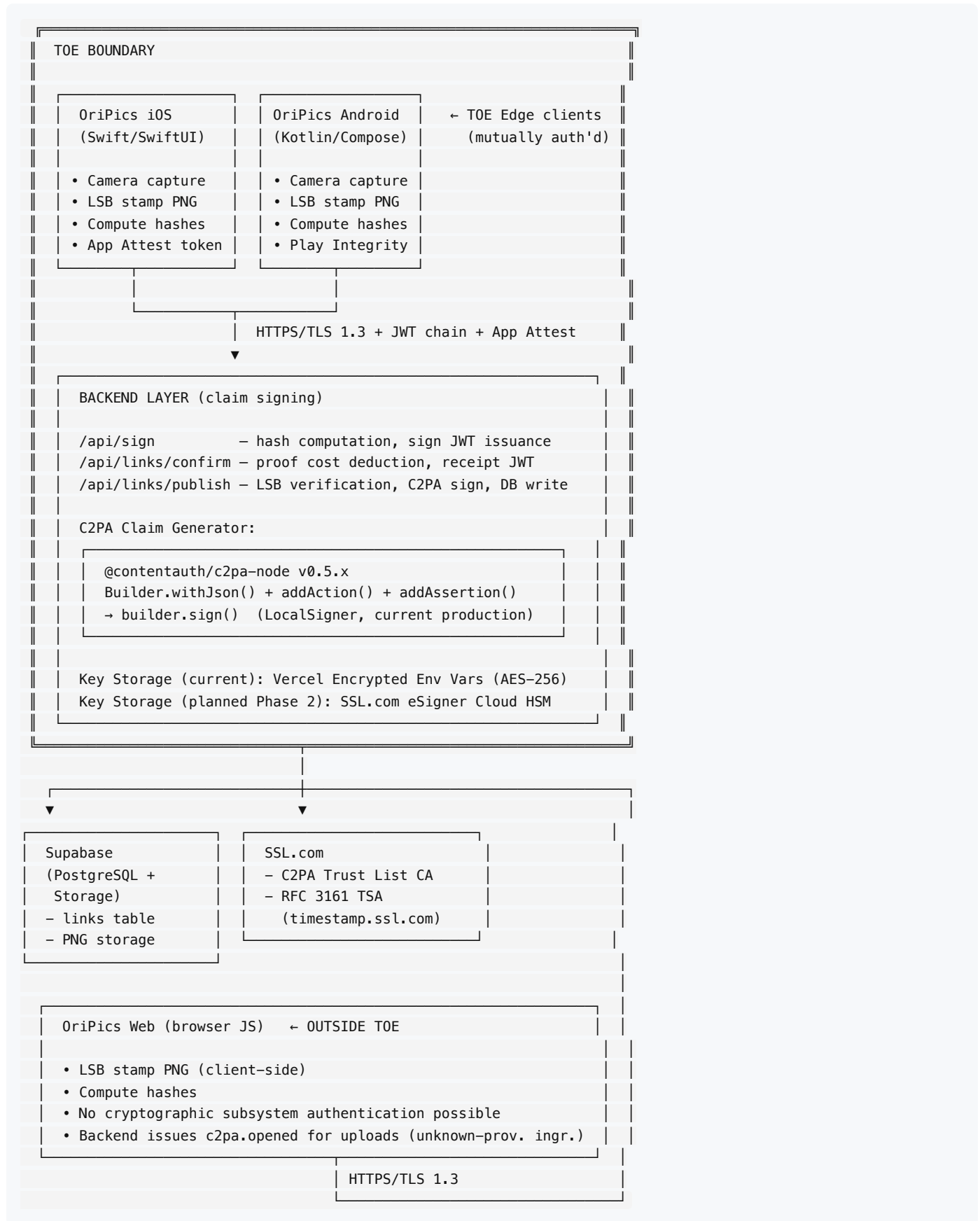
Key features:

- Steganographic pixel-integrity stamping (proprietary LSB embedding + HMAC verification)
- C2PA manifest generation and attachment to output PNG images
- Two trust tiers: Standard (web upload or mobile file pick) and Verified (mobile native camera capture with device attestation)
- Preservation of provenance chain: pre-existing C2PA manifests are included as `c2pa.ingredient` assertions
- Public shareable links with 7-day (Free) or permanent (Pro/Business) retention

Target audience: Individual consumers (B2C, traffic accidents, personal proof) and small businesses (B2B, real estate agencies, media organizations).

C.1.4. Generator Product Target of Evaluation (TOE) Description

The OriPics Generator Product TOE uses a **Distributed Implementation Class**. Claim preparation happens on Edge devices; claim signing happens on the Backend service.



Platform runtime:

- Backend: Vercel Serverless Functions (AWS-backed), Node.js 18+ runtime

- Edge (Web): Browser JavaScript (Chrome, Safari, Firefox — all modern browsers)
- Edge (iOS): Swift/SwiftUI, iOS 16+, ARM64
- Edge (Android): Kotlin/Jetpack Compose, Android 11+, ARM64

Third-party services in TOE scope:

Service	Role	Security Properties
Vercel	Hosting, serverless compute, CDN, TLS termination	SOC 2 Type II, encrypted env vars, immutable deployments
Supabase	PostgreSQL database, object storage (S3-compatible)	SOC 2 Type II, RLS policies, AES-256 encryption at rest
SSL.com	C2PA Trust List CA, eSigner Cloud HSM, TSA	C2PA Trust List registered CA (<code>SSL.com C2PA ECC/RSA Root CA 2025</code>), WebTrust audited
GitHub	Source code repository, CI/CD	Branch protection, Dependabot, CodeQL, secret scanning
Apple App Attest	iOS device integrity attestation (Verified tier)	Apple-managed PKI, attestation key bound to device Secure Enclave
Google Play Integrity	Android device integrity attestation (Verified tier)	Google-managed, Play Integrity API

C.1.5. Implementation Class

Distributed

The OriPics Generator Product is classified as **Distributed** because claim generation spans both Edge and Backend subsystems:

Subsystem	TOE	Role in Claim Generation
Edge — OriPics iOS	In TOE	Native camera capture; steganographic LSB stamping; HMAC hash computation; App Attest token for Verified tier; PNG upload via signed URL
Edge — OriPics Android	In TOE	Same as iOS; Google Play Integrity for Verified tier
Edge — OriPics Web	Outside TOE	Content submission portal only. LSB stamping and hash computation performed client-side, but no cryptographic subsystem authentication possible. Backend ingests all Web uploads with <code>c2pa.opened (no digitalSourceType)</code> + uploaded file as <code>parent0f</code> ingredient of unknown provenance.
Backend — Vercel Functions	In TOE	Receives hashes from Edge via HTTPS; verifies mobile Edge identity via JWT chain + LSB hash extraction + App Attest/Play Integrity; constructs C2PA manifest; signs manifest using <code>@contentauth/c2pa-node</code> ; writes to Supabase

Signing key location: The C2PA signing private key is held exclusively by the Backend. **Current production:** Vercel Encrypted Environment Variables (AES-256). **Planned Phase 2:** SSL.com eSigner Cloud HSM via CSC API (not yet implemented as of v2.2). No signing key material is present on any Edge device.

Claim generator values:

- Standard tier (all platforms): `oripics/0.1.0`
- Verified tier (mobile native capture): `oripics/0.1.0 (mobile-native)`

C.1.6. Target Max Assurance Level

Level 1

C.1.7. Target Generator Product Capabilities

Claim generation (media type OriPics *writes* — the signed output asset):

- Still image media types:
 - `image/png`

OriPics always re-encodes the processed asset to PNG before signing (the steganographic proof layer is embedded in PNG), so every OriPics-generated manifest is carried by an `image/png` asset.

Claim ingestion / validation (media types OriPics *reads* — a prior-manifest asset uploaded by the user and preserved as a `parentOf` ingredient, per §C.2.7):

- Still image media types:
 - `image/png`
 - `image/jpeg`

Users may upload an existing C2PA-bearing asset in either PNG or JPEG; OriPics reads its active manifest (`Reader.fromAsset()`), preserves it as a `c2pa.ingredient.v3` ingredient, and then generates a new PNG output. The `sample-oripics-with-ingredient.png` sample demonstrates ingestion of a JPEG ingredient (a Google Pixel Camera `image/jpeg`), consistent with this declaration.

Claim validation: OriPics is submitted as a **Generator Product**, not a Validator. It additionally operates an internal verification function (`readC2paManifest()` , backing the convenience user-facing `verify` feature) that reads a manifest and reports a trust status. This verifier **consults the C2PA Trust List** (see §C.2.4 §5): it loads the official C2PA Conformance Trust List (signing + TSA CAs) and reports the active manifest signer as `trusted` when its certificate chains to that list. **Scope limitation (pinned SDK):** this internal verifier is bound to `@contentauth/c2pa-node v0.5.x ( c2pa-rs v0.82.1 )`, which does **not** honor a trusted RFC 3161 timestamp when an *ingested third-party ingredient's* per-image certificate is already expired — it reports that ingredient as `signingCredential.expired / untrusted` , whereas a current conformant validator (e.g. `c2patool` with the same Trust List) correctly resolves it to `trusted` (see §C.2.4 *Note on SDK version*). OriPics therefore treats **`c2patool` / `contentcredentials.org` as the authoritative validation reference**, and will align the internal verifier once a conformant newer SDK build is available upstream. This limitation affects only the convenience `verify` feature's reporting of ingested ingredients; it does not affect claim *generation* (the OriPics-signed asset), which is what this Generator Product is assessed on.

C.2. Security Architecture Details

C.2.1. Authentication for Certificate Enrollment

Certificate Enrollment Process

OriPics uses SSL.com as its Certification Authority (CA), which is on the C2PA Trust List. OriPics is a Level 1 (software assurance) Generator Product: the claim signing key pair is generated and held by OriPics, and SSL.com issues the certificate against OriPics's CSR.

Current (pre-production / sandbox):

1. CSR generated with OpenSSL (ECDSA P-256) on a secured workstation
2. CSR submitted to SSL.com `c2pasign.com` sandbox for test cert issuance
3. Test cert + private key stored as Vercel encrypted environment variables
4. LocalSigner in `@contentauth/c2pa-node` uses the cert+key to sign manifests

Production (post-Conformance Letter) — per SSL.com's C2PA certificate issuance process:

1. OriPics generates an ECDSA P-256 key pair and CSR on a secured, network-isolated workstation (DN: `CN=OriPics, O=SantaHades Co., Ltd., C=KR`). The local private-key file is securely deleted after it is loaded into the production secret store (below).
2. OriPics submits, via secure email, (a) the Conformance Letter received from the C2PA Conformance Program and (b) the CSR.
3. **Identity verification:** an officer authorized to legally bind SantaHades Co., Ltd. (the Representative Director & CEO) creates an account on the SSL.com portal and completes SSL.com's AI-based identity check to obtain Individual Validation (IV) approval. (Alternatively, a signed Delegation of Authority letter authorizes a designated requestor.)
4. SSL.com's validation team verifies the request and issues the C2PA Claim Signing Certificate, delivered via the OriPics SSL.com account.
5. OriPics installs the issued certificate and corresponding private key into Vercel encrypted environment variables (Production scope); the `@contentauth/c2pa-node` LocalSigner uses them to sign manifests server-side.

Future enhancement (Phase 2, not evaluated in this submission): Migrate signing-key custody to SSL.com eSigner Cloud HSM (FIPS 140-2 Level 3) via the CSC API, so the private key is generated within and never leaves the HSM, with OriPics authenticating over OAuth 2.0 client credentials. This is a post-conformance hardening step outside the current Level 1 evaluation.

Management of Certificate Enrollment Authentication Secrets

(Required for Assurance Level 1)

Two distinct secret sets are involved:

1. CA portal authentication (enrollment time): Access to the SSL.com portal for identity verification and certificate retrieval uses username + password + 2FA, combined with SSL.com's Individual Validation (IV) identity check. SSL.com stores its C2PA root signing keys in its own hardware HSMs per its CP/CPS (<https://legal.ssl.com/documents/SSLcom-CP-CPS.pdf>).

2. OriPics claim signing private key (runtime):

- Stored as a Vercel encrypted environment variable (`ORIPICS_C2PA_KEY_PEM`), encrypted at rest with AES-256, scoped to the Production environment only.
- Preview/Development use separate sandbox credentials.
- Secrets are never committed to source code; `.env` files are gitignored; GitHub secret scanning is enabled to detect accidental exposure.
- Access to Vercel project settings (where env vars are managed) requires team-owner authentication with MFA enabled; no other team members exist.

C.2.2. Key Generation, Storage, and Usage

Key Generation and Storage Method

(Required for Assurance Level 1)

1. Key generation and storage:

- **Algorithm:** ECDSA with P-256 (ES256), as required by C2PA specification
- **Current production:** Key generated locally via OpenSSL, stored as PEM in Vercel encrypted environment variable (`ORIPICS_C2PA_KEY_PEM`). Vercel encrypts environment variables at rest with AES-256. The key is decrypted at function cold start, held in Lambda memory only for the duration of the signing operation, and not persisted to disk. This satisfies Level 1 Req 6.2.1.1 (encrypted storage).
- **Planned Phase 2 (not yet implemented):** Migrate to SSL.com eSigner Cloud HSM (FIPS 140-2 Level 3). Private key would be generated within the HSM and never leave it. OriPics code would call the CSC API for signing operations. This enhancement is planned post-conformance and is not evaluated in this submission.

2. Key access controls:

- Only the Vercel Functions runtime in the Production environment can access the signing credentials
- Vercel environment variables are encrypted at rest (AES-256) and in transit (TLS 1.3)
- Access to the key PEM in Vercel is restricted to the single Vercel team owner (MFA-protected); no other team members exist
- Application code accesses the key only within the `/api/links/publish` route handler; no other code path reads the signing key
- No signing key material is present on any Edge device (Web browser, iOS, Android)

3. Key rotation process:

Rotation schedule:

- **Regular:** annually (every 365 days)
- **On certificate expiration:** 30 days before expiry
- **Emergency:** immediately upon suspected key compromise

Rotation procedure (consistent with the enrollment process in §C.2.1):

1. Generate a new ECDSA P-256 key pair and CSR on the secured, network-isolated workstation
2. Submit the new CSR to SSL.com and obtain the new certificate via the SSL.com account
3. Install the new cert + key into Vercel encrypted environment variables (Production scope)
4. Verify signing works with the new key (test signing request) before cutover
5. Submit a revocation request for the old certificate to SSL.com
6. Record the rotation in the key rotation log

Because Vercel environment-variable deployment is atomic, rotation is performed with zero downtime. (When the Phase 2 eSigner Cloud HSM model is adopted, key custody and destruction move into the HSM per SSL.com's key management policy.)

4. Edge-to-Backend mutual authentication (Distributed Implementation Class):

TOE scope: Req 6.2.1.5/6 (mutual authentication of Edge and Backend subsystems) applies **only to the mobile Edge clients (iOS, Android)** which are within the TOE. The Web client is explicitly outside the TOE and is not subject to subsystem mutual authentication

requirements. The Backend accepts Web uploads but ingests them with a `c2pa.opened` action (no `digitalSourceType`; uploaded file recorded as a `parent0f` ingredient of unknown provenance) — making no creation or capture claim, consistent with its content-submission-only role.

The OriPics Distributed architecture uses a three-stage JWT chain to authenticate the **mobile** Edge client to the Backend signing service. This mechanism verifies that: (a) the user is authenticated (NextAuth session); and (b) the PNG uploaded to Storage was produced by a legitimate OriPics Edge client (LSB hash verification).

Stage 1 — `/api/sign` (Backend):

1. Validates NextAuth session; checks credit balance
2. Computes `final_hash = HMAC-SHA256(salt, meta_bytes || inner_hash || border_hash)` where `inner_hash` and `border_hash` are computed by the Edge from the original image pixels
3. Issues a sign JWT (`aud: links/confirm`, TTL 5 minutes) containing `{ user_id, tier, link_id, storage_path, timestamp, width, height, final_hash_hex }`
4. Returns `final_hash` (hex) and a Supabase signed upload URL to the Edge

Stage 2 — Edge (all clients):

1. Embeds `meta_bytes || final_hash` into the PNG border pixels via LSB steganography
2. Uploads the stamped PNG to Supabase Storage using the signed upload URL
3. Calls `/api/links/confirm` with the sign JWT

Stage 3 — `/api/links/confirm` (Backend):

1. Verifies sign JWT signature, expiry, and audience
2. Deducts proof credits
3. Issues a receipt JWT (`aud: links/publish`, TTL 30 days) containing all sign JWT claims including `final_hash_hex`

Stage 4 — `/api/links/publish` (Backend):

1. Verifies receipt JWT signature, expiry, and audience
2. Verifies `claims.user_id === sessionUserId` (prevents cross-user replay)
3. Downloads the stamped PNG from Supabase Storage
4. **LSB hash extraction and verification:** Decodes PNG pixels using `pngjs`, reads border-pixel LSBs using `extractPayloadV4()`, extracts embedded `final_hash` from the payload. Compares against `final_hash_hex` from the receipt JWT using `crypto.timingSafeEqual()`. Mismatch → 422 response + credit refund
5. Only if hash matches: constructs and signs the C2PA manifest

This mechanism ensures that a tampered PNG (one where pixel content was swapped after the sign JWT was issued but before publish) will fail LSB verification and be rejected before C2PA signing.

Backend-to-SSL.com communication:

- At runtime (Level 1), signing is performed locally by the `@contentauth/c2pa-node` LocalSigner using the OriPics-held key; the Backend makes **no** per-signing call to SSL.com. The only runtime interaction with SSL.com is RFC 3161 timestamp requests to SSL.com's C2PA-conformant TSA.
- Certificate retrieval occurs at enrollment/rotation time via the SSL.com portal (see §C.2.1).
- All communication between the Backend and SSL.com is over HTTPS/TLS 1.3.
- (Phase 2: if signing-key custody migrates to the eSigner Cloud HSM, the Backend would authenticate per-signing to the CSC API over OAuth 2.0 client credentials. Not part of this submission.)

Mobile Edge device attestation (Verified tier):

- iOS: Edge client obtains an Apple App Attest attestation token (`DCAppAttestService`), bound to the device Secure Enclave key and a server-issued nonce. Backend verifies the attestation chain against Apple's root CA
- Android: Edge client obtains a Google Play Integrity verdict token, bound to the app's Play Store identity and a server-issued nonce. Backend verifies the token against Google's Play Integrity API
- Attestation token hash is stored in the `com.oripics.verified` C2PA assertion for audit

C.2.3. Protections Against Claim Generator Misconfiguration and Abuse

Processes for Detecting Vulnerabilities in Dependencies (SCA/SBOM)

(Required for Assurance Level 1)

1. SCA/SBOM dependency vulnerability scanning tools:

- **GitHub Dependabot:** Enabled on the repository (SantaHades/OriPics-MVP-Front). Monitors all npm dependencies for known CVEs from the GitHub Advisory Database (NIST NVD data). Configured via .github/dependabot.yml :
 - Major version updates: blocked (manual review required)
 - Minor/patch updates: grouped weekly PRs
 - Security updates: automatic PRs for CRITICAL and HIGH severity
- **GitHub CodeQL:** Enabled for JavaScript/TypeScript static analysis. Runs on every PR and weekly scheduled scans. Detects OWASP Top 10 vulnerabilities (SQL injection, XSS, command injection, path traversal, etc.)
- **GitHub Secret Scanning:** Enabled to detect accidental credential commits (API keys, certificates, tokens)
- **npm audit:** Run as part of the development workflow; npm audit checks against the npm advisory database

2. Build and deployment pipeline vulnerability prevention:

- Dependabot automatically creates PRs for CRITICAL and HIGH severity vulnerabilities
- The development team triages vulnerability reports within 7 days of detection
- CRITICAL and HIGH vulnerabilities are fixed or mitigated within 90 days (C2PA Security Requirements §6.3.1)
- Vercel deployments are immutable: each deployment is a frozen snapshot of the code at a specific Git commit. Once deployed, the running code cannot be modified
- GitHub branch protection on main requires PR review and CI passage before merge
- Historical evidence: 21 Dependabot advisories triaged on 2026-05-09; Next.js 14.1.0 → 14.2.35 security patch applied (16 advisories resolved); next-intl prototype pollution fix applied 2026-05-10

C.2.4. Protections Against Misconfiguration and Abuse of Software That Processes or Modifies Digital Content or Assertions

(Required for Assurance Level 1)

The software that processes and/or modifies Digital Content in the OriPics TOE includes:

1. **Client-side steganographic stamping** (oripics-stamp library, runs on Edge): Embeds integrity data into image border pixels via LSB encoding. Hash computation uses crypto.subtle.digest() (Web Crypto API on browser/mobile)
2. **Server-side C2PA manifest construction and signing** (@contentauth/c2pa-node , runs on Backend): Constructs and signs the C2PA JUMBF manifest box appended to the output PNG

1. Action selection and Digital Source Type (DST):

Action selection is determined by two factors: (a) whether a prior C2PA manifest exists in the input asset, and (b) whether the claim was generated from a verified mobile camera capture (Verified tier, within TOE) or a web/file-pick upload (Standard tier, outside or peripheral to TOE):

Scenario	Primary action	digitalSourceType	Ingredient	TOE
Verified tier, no prior manifest (mobile camera, App Attest/Play Integrity confirmed)	c2pa.created	digitalCapture	None	Within TOE
Standard tier, no prior manifest (web upload or mobile file pick)	c2pa.opened	None	uploaded file, parentOf , unknown provenance	Web = outside TOE
Any tier, prior C2PA manifest present	c2pa.opened	None	prior manifest, parentOf	—

c2pa.created asserts creation of the asset itself and is therefore used only for Verified-tier captures, where in-app camera hardware (confirmed via App Attest or Play Integrity) directly produced the asset within the TOE; in that case digitalSourceType: digitalCapture is also asserted. For all ingested files (Standard tier and any upload carrying a prior manifest), OriPics did not create the asset and uses c2pa.opened instead.

C2PA spec / conformance note (per Scott S. Perry, 2026-05-30): A Generator Product must not assert c2pa.created for an asset it did not create — created is a claim about creation of the asset, not merely of the manifest. The only conformant inception action for an ingested file is c2pa.opened , with the ingested file recorded as a parentOf ingredient of unknown provenance (OriPics did not create it and cannot prove C2PA creation provenance). c2pa.opened requires that ingredient; the SDK adds it automatically via builder.setIntent('edit') after builder.addIngredient(... relationship: "parentOf") . (A standalone

`c2pa.published` first action is likewise nonconformant for a parentless manifest, failing with `assertion.action.malformed`.) If creation attribution were ever required, the conformant mechanism would be a CAWG identity assertion in `gathered_assertions`; OriPics makes no creation claim and does not use it.

Implementation: `apps/web/src/lib/oripics-stamp/c2pa.ts` (`attachC2paManifest()`).

2. Assertions and Claim v2 structure:

OriPics uses `@contentauth/c2pa-node v0.5.x` (backed by `c2pa-rs v0.82.1`) with Claim v2.

The C2PA-standard assertions OriPics produces — `c2pa.actions`, `c2pa.ingredient`, and the claim/ingredient thumbnails — are placed in `created_assertions` (attributed to the signer), the conformant location per C2PA Claim v2 spec. This is **not** the SDK default: `c2pa-rs` places only the hard binding (`c2pa.hash.data`) in `created_assertions` and everything else in `gathered_assertions` unless explicitly configured. OriPics configures it via the `c2pa-rs` builder setting:

```
builder.created_assertion_labels = [
  "c2pa.actions", "c2pa.ingredient",
  "c2pa.thumbnail.claim", "c2pa.thumbnail.ingredient"
]
```

(supplied through `createTrustSettings` / `mergeSettings` at sign time; the labels are *base* labels — no version/instance suffix). With this configuration, `c2pa.actions.v2` and `c2pa.ingredient.v3` are emitted in `created_assertions` regardless of whether the action is `c2pa.created` (added via `builder.addAction()`) or `c2pa.opened` (generated via `builder.setIntent('edit')` after `builder.addIngredient(... relationship: "parentOf")`).

The vendor-specific `com.oripics.*` assertions are added via `builder.addAssertion()` and are intentionally left in `gathered_assertions` — they carry OriPics-specific data and are not C2PA-standard assertions.

Assertion label	Claim v2 placement	How added	Content
<code>c2pa.hash.data</code>	<code>created_assertions</code>	Auto-generated by <code>c2pa-rs</code>	Cryptographic hash binding
<code>c2pa.actions.v2</code>	<code>created_assertions</code>	<code>addAction()</code> / <code>setIntent('edit')</code> + <code>created_assertion_labels</code>	<code>c2pa.created</code> (Verified capture) or <code>c2pa.opened</code> (Standard upload / prior manifest) + platform action
<code>c2pa.ingredient.v3</code>	<code>created_assertions</code>	<code>addIngredient()</code> + <code>created_assertion_labels</code>	<code>parentOf</code> ingredient (prior manifest, or uploaded file of unknown provenance)
<code>c2pa.thumbnail.claim</code> / <code>c2pa.thumbnail.ingredient</code>	<code>created_assertions</code>	Auto-generated + <code>created_assertion_labels</code>	Claim / ingredient thumbnails
<code>com.oripics.proof</code>	<code>gathered_assertions</code>	<code>builder.addAssertion()</code>	Tier, <code>link_id</code> , <code>verify_url</code> , <code>stamp_version</code> , dimensions, optional GPS
<code>com.oripics.verified</code>	<code>gathered_assertions</code>	<code>builder.addAssertion()</code>	Platform, <code>attest_token_hash</code> , <code>device_integrity</code> (<i>Verified tier only</i>)

3. SCA/SBOM scanning for all content-processing software:

- Same tools as §C.2.3: GitHub Dependabot + CodeQL + npm audit cover all npm dependencies including `@contentauth/c2pa-node` and `pngjs`
- `@contentauth/c2pa-node` is the official C2PA SDK maintained by the Content Authenticity Initiative (CAI). Security updates are tracked via Dependabot
- `pngjs` (used for server-side LSB extraction) is monitored by Dependabot
- Vercel's immutable deployment model ensures that a specific deployment always runs the exact dependency versions present at build time

4. Vulnerability remediation pipeline:

- Same 90-day CRITICAL/HIGH remediation policy as §C.2.3

- The `@contentauth/c2pa-node` package includes native binary addons (Rust-compiled); Dependabot monitors its npm advisory entries

5. C2PA Trust List consumption in the verification function (validation logic):

The OriPics verification function (`readC2paManifest()` in `apps/web/src/lib/oripics-stamp/c2pa.ts` , backing the public verify feature and the `/api/verify` and `/api/links/[id]/c2pa` routes) loads the **official C2PA Trust List** when validating a manifest's signature, so that a certificate chaining to the Trust List yields a `trusted` verdict:

- **Trust anchors:** the C2PA Conformance Trust List — `C2PA-TRUST-LIST.pem` (signing CAs) combined with `C2PA-TSA-TRUST-LIST.pem` (timestamp CAs) from the `c2pa-org/conformance-public` repository — is bundled in the product (`apps/web/src/lib/oripics-stamp/c2pa-trust-list.ts` ; refreshed via `scripts/update-c2pa-trust-list.mjs`) and supplied to `createTrustSettings({ verifyTrustList: true, trustAnchors, trustConfig })` .
- **Verification flags:** `createVerifySettings({ verifyTrust: true, verifyTimestampTrust: true })` — the signing certificate is validated against the Trust List, and a cert that is expired at validation time but was within validity at the trusted-timestamp time is accepted on that basis.
- **Trust configuration:** the EKU allow-list (`store.cfg`) constrains acceptable extended key usages to the C2PA-recognized set.
- A signing certificate chaining to a Trust List CA (e.g. `SSL.com C2PA ECC/RSA Root CA 2025` , the OriPics production CA) is reported `trusted` ; the current sandbox/dev certificate is self-signed and therefore reported `untrusted` (expected until production credentials are issued).
- In self-signed sandbox mode (`ORIPICS_C2PA_CA_PEM` set), the dev CA is used as the trust anchor instead; production mode (no `ORIPICS_C2PA_CA_PEM`) uses the C2PA Trust List above.

Note on SDK version: OriPics pins `@contentauth/c2pa-node v0.5.x` (c2pa-rs v0.82.1). When this version validates an ingested third-party manifest whose per-image certificate is already expired (e.g. a Google Pixel Camera ingredient), it reports `signingCredential.expired` even though the manifest carries a trusted timestamp proving validity at signing time; conformant validators (e.g. current `c2patool`) resolve such a manifest to `trusted` . This affects only the *ingredient's* status; the OriPics active manifest is validated independently.

C.2.5. Protections Against Interception and/or Modification of Traffic

Encryption of Network Traffic

(Required for Assurance Level 1 for Distributed Implementation Class)

TLS versions and cryptographic protocols:

Communication Channel	Protocol	Notes
Edge (all) → Vercel Backend (API calls)	TLS 1.3	Enforced by Vercel CDN/Edge; HTTPS redirect enforced
Edge (all) → Supabase Storage (PNG upload via signed URL)	TLS 1.3	Supabase enforced
Backend → Supabase (DB + Storage)	TLS 1.3	Enforced via <code>tls.DEFAULT_MIN_VERSION = 'TLSv1.3'</code> in <code>src/instrumentation.ts</code> ; Supabase endpoints support TLS 1.3
Backend → SSL.com TSA (timestamp.ssl.com)	TLS 1.3	SSL.com enforced; RFC 3161 timestamp requests
Backend → SSL.com eSigner CSC API	TLS 1.3	SSL.com enforced — Phase 2 only (not used in current Level 1 model)

- **All network communication** between subsystems is encrypted with TLS 1.3
- Vercel enforces TLS 1.3 for all inbound connections; HTTP is automatically redirected to HTTPS
- No plaintext HTTP communication is used between any subsystems
- The `final_hash_hex` transmitted in the JWT chain is protected by TLS in transit and by the HMAC-SHA256 JWT signature at rest

C.2.6. Protections Against Exploitation of Hosting Environment

(Required for Assurance Level 1 for Distributed Implementation Class)

1. IAM system and security boundaries:

- **Vercel:** Team-based RBAC. The OriPics Vercel project is owned by a single team with one owner (Yongseog Son). MFA is required for the team owner account. Vercel enforces isolation between projects and between customers at the infrastructure level (SOC 2 Type II certified)
- **Supabase:** Row-Level Security (RLS) is enabled on all tables. The application uses `service_role` key for server-side operations (never exposed to Edge clients). Edge clients use `anon` key with RLS policies restricting access to authenticated users' own data
- **GitHub:** Repository is private. Branch protection on `main` requires PR approval and CI passage. Force push is disabled
- **Edge clients:** No signing keys, no backend secrets, no `service_role` or `anon` keys embedded in Edge code. Edge clients authenticate users via NextAuth session cookies; all privileged operations require a valid session

2. Access policies for human and non-human principals:

Principal	Access	MFA
Vercel team owner (human)	Full project access, env var management	Yes
Vercel Functions runtime (non-human)	Read env vars, execute functions	N/A
Supabase <code>service_role</code> (non-human)	Full database + storage access	N/A (API key, server-only)
Supabase <code>anon</code> key (non-human)	RLS-restricted read/write	N/A (API key + user JWT)
GitHub repository admin (human)	Source code, CI/CD, secrets	Yes
Dependabot (non-human)	Create PRs for dependency updates	N/A (GitHub-managed)
Edge client / end user (non-human)	NextAuth session only; no infrastructure access	N/A

3. IAM policies for main cloud resources:

- **Vercel Functions:** Only deployed code from the `main` branch can execute in Production. Preview deployments run in isolated environments with separate env vars
- **Supabase Storage** (`oripics-proofs` bucket): Accessed only via `service_role` key from Backend Functions. Signed upload URLs for Edge clients have a 5-minute TTL and are path-restricted to the specific PNG being uploaded. No public write access; public read is restricted to published links only
- **Supabase PostgreSQL:** All tables have RLS enabled. `service_role` bypasses RLS for server-side operations; `anon` key is RLS-restricted

4. Vulnerability scanning and security review:

- GitHub Dependabot + CodeQL + Secret Scanning (as described in §C.2.3)
- OWASP Top 10 coverage via CodeQL rules
- Vercel and Supabase perform their own infrastructure-level security management as part of their respective SOC 2 Type II programs

5. Vulnerability remediation process:

- CRITICAL: Target fix within 30 days
- HIGH: Target fix within 90 days
- MODERATE: Target fix within 180 days
- LOW: Tracked and addressed on a best-effort basis
- Dependabot automatically creates PRs; team triages within 7 days
- Evidence of historical remediation: 21 advisories triaged 2026-05-09, Next.js security patch applied same day

C.2.7. Capture-Only Content Policy and Pre-Existing Manifest Handling

Content Policy and TOE Boundary

OriPics is a **photo provenance platform**. The `digitalCapture` DST is asserted only when the Generator Product can cryptographically confirm that the asset was captured by a hardware camera under the control of an authenticated device:

Client	Tier	TOE	Action	DST	Ingredient
OriPics Web	Standard	Outside TOE	<code>c2pa.opened</code>	None	uploaded file, <code>parent0f</code> , unknown provenance
OriPics iOS	Standard (file pick)	In TOE (device auth'd)	<code>c2pa.opened</code>	None	uploaded file, <code>parent0f</code> , unknown provenance

Client	Tier	TOE	Action	DST	Ingredient
OriPics iOS	Verified (camera capture)	In TOE (App Attest)	c2pa.created	digitalCapture	None
OriPics Android	Standard (file pick)	In TOE (device auth'd)	c2pa.opened	None	uploaded file, parentOf , unknown provenance
OriPics Android	Verified (camera capture)	In TOE (Play Integrity)	c2pa.created	digitalCapture	None

c2pa.created (asserting creation of the asset) is issued only for Verified-tier captures, where device attestation (App Attest / Play Integrity) confirms in-app camera hardware capture; digitalCapture accompanies it. All file-pick / Web uploads are ingested via c2pa.opened because OriPics did not create the asset. OriPics does not certify AI-generated images, screenshots, or composites.

Pre-Existing C2PA Manifest Handling

When an uploaded image contains a pre-existing C2PA manifest (e.g., a photo previously signed by another C2PA generator product such as a Pixel Camera, Leica M11-P, or another conformant tool), OriPics detects this using @contentauth/c2pa-node Reader.fromAsset() before constructing the new manifest.

Implemented: Approach A — Ingredient (parentOf)

OriPics implements **Approach A**: every ingested file is recorded as a c2pa.ingredient assertion (relationship: "parentOf") in the new OriPics manifest, under a c2pa.opened action. When a pre-existing C2PA manifest is detected, that manifest is preserved in the ingredient, maintaining the full provenance chain; when none is present, the ingredient simply records the uploaded file as unknown provenance.

Implementation (apps/web/src/lib/oripics-stamp/c2pa.ts):

- 1. Before constructing the new manifest, Reader.fromAsset() attempts to read the active manifest from the input PNG
- 2. **Verified tier** (mobile native camera, App Attest / Play Integrity confirmed): OriPics created the asset via attested hardware capture, so the primary action is c2pa.created with digitalSourceType: digitalCapture ; no ingredient is added
- 3. **All ingested files** (Standard-tier Web/file-pick uploads, and any upload — verified or not — that already carries a prior C2PA manifest): OriPics did not create the asset, so the primary action is c2pa.opened (C2PA spec §9.3.2) and the ingested file is recorded as a parentOf ingredient via builder.addIngredient(... relationship: "parentOf") + builder.setIntent('edit') :
 - **Prior C2PA manifest present**: the ingredient preserves that manifest, maintaining the full provenance chain
 - **No prior manifest** (plain upload): the ingredient has **unknown provenance** — OriPics neither created it nor can prove its C2PA provenance

Scenario	Action	Ingredient
Verified tier, mobile camera, no prior C2PA	c2pa.created + digitalCapture DST	None
Standard tier, web/file-pick, no prior C2PA	c2pa.opened (no DST)	uploaded file, parentOf , unknown provenance
Any tier, prior C2PA manifest present	c2pa.opened (no DST)	c2pa.ingredient.v3 with relationship: "parentOf" (prior manifest preserved)

(Note on Verified tier): Mobile native camera captures (Verified tier) produce fresh in-app captures with no prior C2PA manifest and are the only case where OriPics asserts c2pa.created . Every other path ingests an existing file via c2pa.opened .

Attachments

The following sample output files are provided with this submission:

- 1. **sample-oripics-standard.png** — Sample PNG with OriPics C2PA manifest (Standard tier, sandbox cert). Action: c2pa.opened (no digitalSourceType); the uploaded file is recorded as a parentOf ingredient of **unknown provenance**. c2pa.actions.v2 , c2pa.ingredient.v3 , and the thumbnails are in created_assertions . validation_state: Valid (only the expected sandbox signingCredential.untrusted notice).

2. **sample-oripics-standard-manifest.json** — Extracted C2PA manifest JSON from the above sample.
3. **sample-oripics-with-ingredient.png** — Sample PNG demonstrating §C.2.7 ingestion of a prior-manifest file: a Google Pixel Camera image (PXL_20250708_194721212.MP.jpg from the C2PA Interoperability Testing Library) processed by OriPics. Action: c2pa.opened ; the prior Pixel Camera manifest is preserved as c2pa.ingredient.v3 with relationship: "parentOf" . Validated with c2patool 0.26.56 + the C2PA Conformance Trust List: the ingested Pixel ingredient is signingCredential.trusted + timeStamp.trusted (its short-lived per-image certificate is accepted via its trusted timestamp); the OriPics active manifest's only failure is the expected sandbox signingCredential.untrusted .
4. **sample-oripics-with-ingredient-manifest.json** — Extracted C2PA manifest JSON from the above sample.

(Note: Trust verdicts are determined by the validator's trust list, not embedded in the file. The verdicts above were produced with c2patool 0.26.56 configured with the C2PA Conformance Trust List — C2PA-TRUST-LIST.pem + C2PA-TSA-TRUST-LIST.pem from c2pa-org/conformance-public — via --settings . Samples are signed with a sandbox test certificate, so the OriPics active manifest is untrusted (expected); when the SSL.com production certificate — a CA on the Trust List — is issued, the active manifest is trusted .)